

```
"""
```

```
Abstract base class for all sensor nodes in the Embedded Sensor Network  
Data Aggregation Hub.
```

```
This module defines :class:`SensorNode`, the common contract that every  
concrete sensor type (temperature, pressure, vibration, gas  
concentration, flow rate) must implement, regardless of which simulated  
transport protocol (MQTT, Modbus, CAN Bus) it communicates over.
```

```
"""
```

```
import re
```

```
from abc import ABC, abstractmethod
```

```
from src.exceptions import CalibrationError, SensorOfflineError
```

```
# Matches IDs like "SN-TEMP-01": literal "SN-", 4 uppercase letters,  
# a hyphen, then a 2-digit number.
```

```
_SENSOR_ID_PATTERN = re.compile(r"^SN-[A-Z]{4}-\d{2}$")
```

```
class SensorNode(ABC):
```

```
    """
```

```
    Abstract base class representing a single physical sensor node.
```

```
    Subclasses must implement the abstract ``sensor_id`` property and the  
    abstract :meth:`read` method. ``SensorNode`` itself cannot be  
    instantiated.
```

```
    Attributes:
```

```
        location (str): Human-readable physical location of the sensor
```

```
            (e.g. "Building A - Boiler Room").
```

```
        unit (str): Unit of measurement for readings (e.g. "C", "kPa").
```

```
        safe_min (float): Lower bound of the sensor's safe operating range.
```

```
        safe_max (float): Upper bound of the sensor's safe operating range.
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        sensor_id: str,
```

```
        location: str,
```

```
        unit: str,
```

```
        safe_min: float,
```

```
        safe_max: float,
```

```
        calibration_offset: float = 0.0,
```

```

):
    """
    Initialize a sensor node.

    Args:
        sensor_id: Unique sensor identifier matching the pattern
            ``SN-TTTT-NN``, where ``TTTT`` is a 4-letter type code
            (e.g. ``TEMP``, ``PRES``) and ``NN`` is a 2-digit number
            (e.g. ``SN-TEMP-01``).
        location: Human-readable physical location of the sensor.
        unit: Unit of measurement for readings (e.g. "C", "kPa").
        safe_min: Lower bound of the sensor's safe operating range.
        safe_max: Upper bound of the sensor's safe operating range.
        calibration_offset: Initial calibration offset applied to raw
            readings. Defaults to ``0.0``. Must be within  $\pm 10\%$  of the
            sensor's full scale (``safe_max - safe_min``).

    Raises:
        ValueError: If ``sensor_id`` does not match the required
            pattern.
        CalibrationError: If ``calibration_offset`` falls outside
             $\pm 10\%$  of the sensor's full scale.
    """
    self._validate_sensor_id(sensor_id)
    self._sensor_id = sensor_id
    self.location = location
    self.unit = unit
    self.safe_min = safe_min
    self.safe_max = safe_max

    # Set via the property so calibration validation applies to the
    # initial value too.
    self._calibration_offset = 0.0
    self.calibration_offset = calibration_offset

    @staticmethod
    def _validate_sensor_id(sensor_id: str) -> None:
        """
        Validate that a sensor ID matches the required ``SN-TTTT-NN`` format.

        Args:
            sensor_id: The sensor ID string to validate.

        Raises:
            ValueError: If ``sensor_id`` is not a string, or does not
                match the pattern ``SN-TTTT-NN`` (4 uppercase letters
                followed by a 2-digit number, e.g. ``SN-TEMP-01``).

```

```

"""
if not isinstance(sensor_id, str) or not _SENSOR_ID_PATTERN.match(
    sensor_id
):
    raise ValueError(
        f"Invalid sensor_id {sensor_id!r}: must match pattern "
        "'SN-TTTT-NN' (4-letter type code, hyphen, 2-digit "
        "number), e.g. 'SN-TEMP-01'."
    )

@property
@abstractmethod
def sensor_id(self) -> str:
    """
    str: The unique identifier of this sensor node.

    Must be implemented by concrete subclasses, typically by
    returning the value stored during :meth:`__init__`
    (``self._sensor_id```).
    """
    raise NotImplementedError

@property
def calibration_offset(self) -> float:
    """
    float: The current calibration offset applied to raw readings.

    Setting this property validates that the new offset is within
    ±10% of the sensor's full scale (``safe_max - safe_min```); an
    out-of-range value raises :class:`CalibrationError`.
    """
    return self._calibration_offset

@calibration_offset.setter
def calibration_offset(self, offset: float) -> None:
    """
    Validate and set the calibration offset.

    Args:
        offset: The proposed calibration offset.

    Raises:
        CalibrationError: If ``abs(offset)`` exceeds 10% of the
            sensor's full scale (``safe_max - safe_min```).
    """
    full_scale = abs(self.safe_max - self.safe_min)
    max_allowed = 0.10 * full_scale

```

```

    if abs(offset) > max_allowed:
        raise CalibrationError(
            self._sensor_id,
            f"calibration_offset {offset} exceeds allowed range of "
            f"+/-{max_allowed} (10% of full scale {full_scale}).",
        )

    self._calibration_offset = offset

def calibrate(self, offset: float) -> None:
    """
    Apply a new calibration offset to this sensor.

    This is a thin convenience wrapper around the ``calibration_offset``
    property setter, so the same validation rules apply.

    Args:
        offset: The new calibration offset to apply.

    Raises:
        CalibrationError: If ``offset`` is outside the allowed
            ±10% of full scale.
    """
    self.calibration_offset = offset

def self_test(self) -> bool:
    """
    Run a basic internal consistency check on this sensor node.

    The default implementation verifies that the current calibration
    offset is still within the allowed ±10% of full scale. Subclasses
    may override this to add additional, sensor-type-specific checks
    (e.g. polling hardware state), and may raise
    :class:`SensorOfflineError` to simulate a sensor that fails to
    respond during the test.

    Returns:
        bool: ``True`` if the sensor passes the consistency check.

    Raises:
        SensorOfflineError: If the sensor's internal state is
            inconsistent enough to be treated as a failed self-test
            (e.g. calibration offset has drifted out of range).
    """
    full_scale = abs(self.safe_max - self.safe_min)
    max_allowed = 0.10 * full_scale

```

```

        if abs(self._calibration_offset) > max_allowed:
            raise SensorOfflineError(
                self._sensor_id,
                "Self-test failed: calibration_offset "
                f"{self._calibration_offset} is out of the allowed "
                f"+/-{max_allowed} range.",
            )

    return True

@abstractmethod
def read(self) -> "SensorReading":
    """
    Take a reading from this sensor.

    Must be implemented by concrete subclasses, including whatever
    transport-specific logic (MQTT, Modbus, CAN Bus) is needed to
    obtain a raw value, then apply ``calibration_offset`` and wrap
    the result in a ``SensorReading``.

    Returns:
        SensorReading: The resulting sensor reading.

    Raises:
        SensorOfflineError: If the sensor cannot be reached.
        CommunicationTimeoutError: If the read times out.
    """
    raise NotImplementedError

def __str__(self) -> str:
    """Return a concise, human-readable description of this sensor."""
    return (
        f"{self.__class__.__name__} {self._sensor_id} "
        f"@ {self.location} ({self.unit}, "
        f"range [{self.safe_min}, {self.safe_max}])"
    )

def __repr__(self) -> str:
    """Return an unambiguous, debugger-friendly representation."""
    return (
        f"{self.__class__.__name__} ("
        f"sensor_id={self._sensor_id!r}, "
        f"location={self.location!r}, "
        f"unit={self.unit!r}, "
        f"safe_min={self.safe_min!r}, "
        f"safe_max={self.safe_max!r}, "
    )

```

```
        f"calibration_offset={self._calibration_offset!r})"  
    )
```